SMART PRODUCT PRICE
COMPARISON SYSTEM

Practical Lab File

Subject	Software Engineering / Web Technology
Academic Year	2024 – 2025
Tech Stack	HTML5, CSS3, Bootstrap 5, PHP, MySQL, XAMPP

Student Name	_____
Enrollment No.	_____
Branch / Semester	_____
College Name	_____
Faculty / Guide	_____

INDEX

PRACTICAL 1

Project Definition & Requirement Engineering

1.1 Project Title

Smart Product Price Comparison System

1.2 Project Definition

The Smart Product Price Comparison System is a web-based application that allows users to search for any product and instantly view its prices, ratings, and availability from multiple e-commerce platforms such as Amazon, Flipkart, Myntra, and Ajio — all on a single comparison page. Instead of manually visiting multiple websites, users get a side-by-side comparison table in one place, making it easy to identify the best deal.

For the academic version, live API connections to marketplaces are not used. Instead, marketplace data is stored in a MySQL database (simulated dummy data) and retrieved via PHP. The system is built using HTML5, CSS3, Bootstrap 5, PHP, and MySQL on XAMPP.

1.3 Problem Statement

Online shoppers frequently face the following challenges:

- They must visit multiple e-commerce websites individually to compare prices for the same product.
- The process is time-consuming — opening multiple tabs, comparing manually, and remembering prices.
- Prices change frequently; there is no single place showing real-time comparisons.
- Users often miss better deals because they do not check all platforms.

The Smart Product Price Comparison System solves all of these problems by providing a centralized comparison dashboard where users search once and see all platform prices, ratings, and availability in a structured table.

1.4 Objectives

1. Allow users to register, log in, and search for products by name or category.
2. Display product prices from multiple marketplaces (Amazon, Flipkart, Myntra, Ajio) in one comparison table.
3. Show product rating and availability (In Stock / Limited / Out of Stock) per platform.
4. Provide filters: sort by lowest price, highest rating, or filter by specific platform.
5. Highlight the best deal (lowest price) automatically in the comparison table.
6. Provide a direct 'Buy Now' link to the platform page for each product listing.
7. Allow users to save products to a personal wishlist for later comparison.
8. Enable admin to manage products, marketplace entries, and price data via an admin panel.

1.5 System Actors & Roles

Actor	Role & Responsibilities
User (Shopper)	Register/Login, Search products by name/category, View comparison table, Apply filters, Click Buy Now link, Save to wishlist

Admin	Login to admin panel, Add/Edit/Delete product entries, Update marketplace price data, Manage platform list, View usage reports
System	Process search query, Query DB for product and price data, Apply filters/sort, Render comparison table, Return best deal highlight

1.6 Key Features

- Product search by name or category with instant results
- Side-by-side price comparison table: Platform | Price | Rating | Availability | Buy Link
- Best Deal Highlight — lowest price platform is highlighted in green
- Filter by: Lowest Price, Highest Rating, Specific Platform (Amazon/Flipkart/etc.)
- Wishlist — save products for comparison later
- Admin Panel — full CRUD on products, prices, and marketplace entries
- Responsive UI using Bootstrap 5 (works on mobile and desktop)
- Direct 'Buy Now' links per platform in the comparison table

1.7 Example System Flow

Step	Action	Component
1	User logs in and types 'Nike Shoes' in the search box	Browser / PHP Auth
2	System searches products table: SELECT WHERE product_name LIKE '%Nike%'	PHP + MySQL
3	System fetches all price rows for matching product_ids from product_prices table	Comparison Engine
4	Results sorted by price (ASC); best deal highlighted; comparison table rendered	PHP + Bootstrap UI
5	User applies 'Filter: Amazon only' — table refreshes showing Amazon rows only	JavaScript + PHP
6	User clicks 'Buy Now' on Flipkart row — redirected to Flipkart product URL	Browser redirect
7	User clicks 'Save to Wishlist' — product added to their wishlist in DB	PHP + MySQL

1.8 Requirement Engineering Stages

Stage 1 – Feasibility Study

Technical: feasible using PHP + MySQL + XAMPP — no licensing cost. Live API calls to Amazon/Flipkart are outside scope; simulated DB data is sufficient for academic demonstration. Economic: free tools; deployable on shared hosting under ₹500/year if needed.

Stage 2 – Requirement Elicitation

Requirements gathered by: observing how online shoppers compare products, surveying classmates about price-checking habits, studying existing comparison sites (Google Shopping, PriceDekho), and faculty guidance on academic scope and tech stack.

Stage 3 – Requirement Specification

All requirements formally documented in SRS (Practical 3) covering 6 functional requirement groups and 7 non-functional requirements including performance, security, and usability.

Stage 4 – Requirement Validation

Requirements reviewed with faculty and team members to confirm completeness, consistency, and feasibility. Use case walkthrough performed with a non-technical user to validate usability of the comparison table concept.

PRACTICAL 2

SDLC Model – Spiral Model

2.1 Introduction to SDLC

The Software Development Life Cycle (SDLC) is a structured framework that governs the planning, design, development, testing, and delivery of software. Choosing the right SDLC model ensures quality, manages risk, and keeps the project on schedule. Common models include Waterfall, Incremental, Agile, Prototype, and the Spiral Model.

2.2 The Spiral Model

The Spiral Model, proposed by Barry Boehm (1986), is a risk-driven iterative development framework. Development proceeds in spiral loops — each loop passing through four quadrants: Planning, Risk Analysis, Engineering, and Evaluation. The model is ideal for projects where requirements evolve, risk is moderate, and iterative feedback improves the final product.

2.3 Four Quadrants Applied to This Project

Quadrant	Core Activity	Applied in This Project
1. Planning	Define iteration objectives, alternatives, constraints	Define product schema, marketplace data structure, UI wireframes, DB design for each cycle
2. Risk Analysis	Identify risks, prototype solutions, assess alternatives	Risk: live API unavailable → use simulated DB data; Risk: slow queries → add DB indexes
3. Engineering	Design, code, and test the iteration deliverable	Build search module → comparison engine → filters → admin panel
4. Evaluation	Stakeholder review; refine scope for next iteration	Faculty/peer review of comparison table output; feedback shapes next module

2.4 Spiral Model Diagram

The diagram shows four development iterations (Cycles 1–4) spiraling outward through the four quadrants. Each cycle produces a working deliverable that feeds into the next cycle.

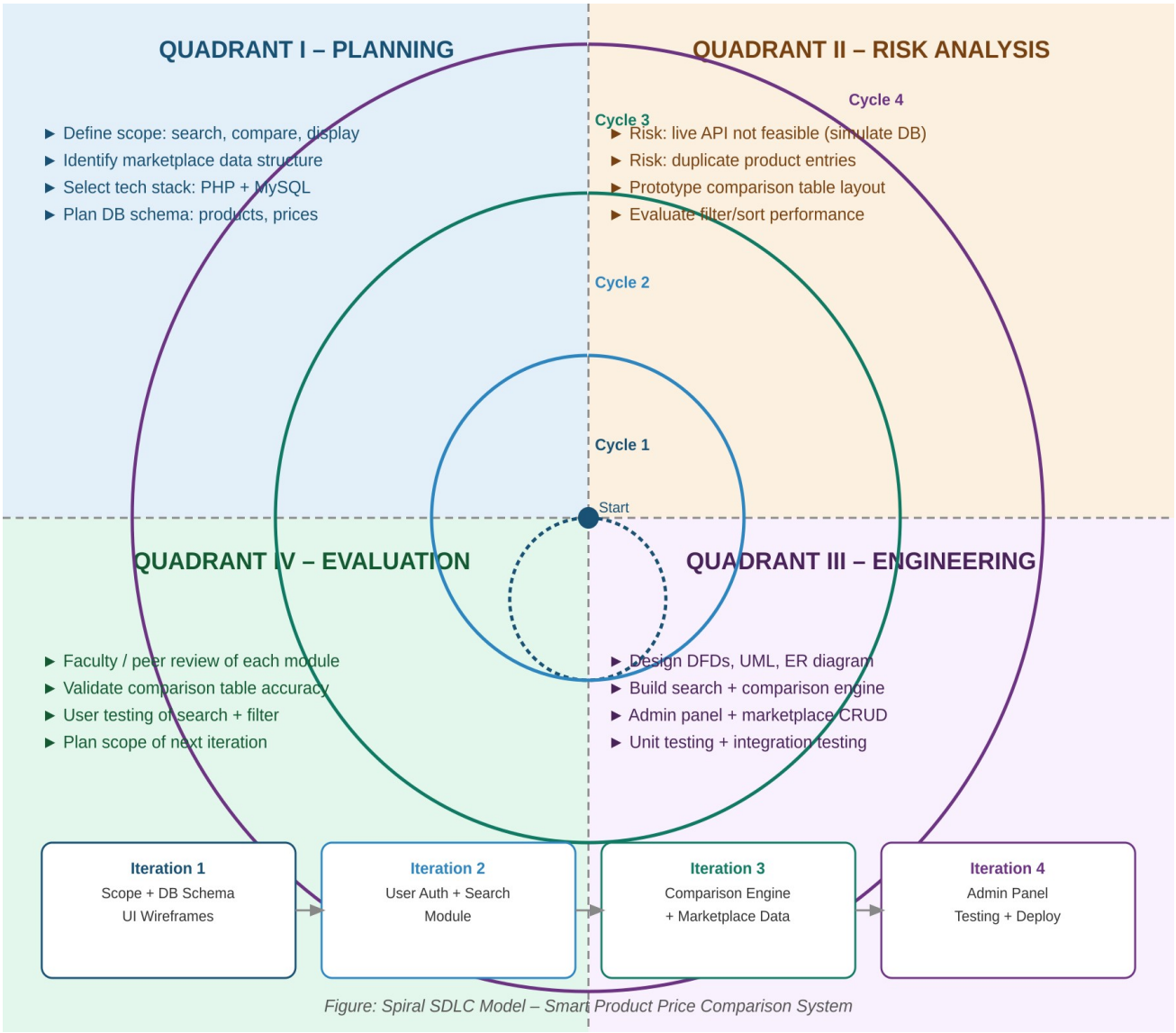

Figure: Spiral SDLC Model – Smart Product Price Comparison System

Figure 2.1 – Spiral SDLC Model for Smart Product Price Comparison System

2.5 Spiral Iterations for This Project

Iteration	Focus	Key Deliverable
Cycle 1	Scope definition, DB schema design, UI wireframes	ER diagram, product/price table design, homepage mockup
Cycle 2	User auth + product search + basic results display	Login/register working; search returns matching products
Cycle 3	Comparison engine + price fetching + filters + sort	Full comparison table with best deal highlight; filter/sort working
Cycle 4	Admin panel + wishlist + full testing + deployment	Admin CRUD; wishlist; 20 test cases executed; deployed on XAMPP

2.6 Justification – Why Spiral?

Model	Key Strength	Why Not Ideal for This Project
Waterfall	Simple, sequential documentation	Rigid — no room to refine comparison table design based on feedback
Agile	Very fast iterations, flexible	Less formal documentation; hard to map to

		academic practicals
Incremental	Module-wise delivery	Lacks formal risk analysis at each stage
Spiral ✓	Risk-driven; handles evolving requirements; structured iterations	BEST FIT — handles API limitation risk; allows iterative refinement of comparison logic

PRACTICAL 3

Software Requirements Specification (SRS)

3.1 Introduction

This SRS document formally defines all requirements for the Smart Product Price Comparison System. It is the authoritative reference for the development team, testers, and faculty throughout the project lifecycle.

System Name	Smart Product Price Comparison System
Version	1.0
Tech Stack	HTML5, CSS3, Bootstrap 5, PHP, MySQL, XAMPP

3.2 System Overview

The system has two user roles: Registered User and Admin. Users interact via a web browser. PHP processes all backend logic. MySQL stores products, prices, marketplaces, users, and wishlists. The comparison engine fetches price data per product from all available marketplace entries and presents them in a sortable, filterable table. Bootstrap 5 ensures a fully responsive interface.

3.3 Assumptions & Dependencies

- Marketplace price data is stored in the database as simulated dummy data (no live API integration).
- Admin is responsible for keeping price data up to date in the DB.
- All users must have a valid email address to register.
- Internet or intranet access is available; modern browser required (Chrome, Firefox, Edge).
- System deployed on XAMPP locally or a PHP 7.4+ compatible web host.
- Product availability (In Stock / Limited / Out of Stock) is stored as a string field per price row.

3.4 Functional Requirements

FR-01: User Authentication

- The system shall allow users to register with username, email, and password.
- The system shall authenticate users via email + bcrypt-hashed password.
- The system shall maintain session state after login and clear it on logout.

FR-02: Product Search

- The system shall allow users to search products by name (partial match using LIKE) or by category.
- The system shall display a list of matching products with name, brand, and category.
- The system shall show a 'No products found' message when no matches exist.

FR-03: Price Comparison Engine

- For a selected product, the system shall fetch all rows from product_prices WHERE product_id matches.
- The system shall JOIN with the marketplace table to display platform names.
- The system shall highlight the row with the lowest price as 'Best Deal'.

FR-04: Filters and Sorting

- The system shall allow sorting the comparison table by: Lowest Price, Highest Rating.
- The system shall allow filtering results to show only a specific platform.
- Filters shall apply without page reload (AJAX or PHP GET parameter).

FR-05: Wishlist

- Users shall be able to save any product to their personal wishlist.
- Users shall be able to view and remove items from their wishlist.

FR-06: Admin Module

- Admin shall manage products: Add (name, brand, category, description), Edit, Delete.
- Admin shall manage product_prices: Add/Edit/Delete price rows per product per marketplace.
- Admin shall manage the marketplace list (add/remove platforms).
- Admin shall view system usage reports: most-searched products, total users, total comparisons.

3.5 Non-Functional Requirements

Category	Requirement
Performance	Comparison table must load within 2 seconds for up to 500 concurrent users on a standard server.
Security	Passwords stored as bcrypt hashes; all DB queries use PDO prepared statements; XSS prevented via htmlspecialchars().
Usability	Comparison table must be readable on mobile (320px) and desktop; filter/sort must be self-explanatory.
Reliability	System must handle empty search results gracefully; all DB errors must be caught and logged.
Scalability	DB must handle 10,000+ product entries and 50+ marketplace price rows per product without degradation.
Maintainability	PSR-12 PHP standards; modular file structure; inline documentation on all functions.
Portability	Compatible with PHP 7.4+ on XAMPP, WAMP, and cPanel shared hosting.

3.6 External Interface Requirements

User Interface

Responsive Bootstrap 5 UI. Search bar prominent on dashboard. Comparison table with alternating row colors, best deal highlighted in green. Filter/sort controls above the table. 'Buy Now' button per platform row opens platform URL in new tab.

Database Interface

MySQL accessed via PHP PDO prepared statements. Five tables: users, products, marketplace, product_prices, wishlist. Foreign keys enforced with ON DELETE CASCADE. Index on product_prices.product_id and product_prices.marketplace_id for fast joins.

Hardware Interface

Any device with a modern browser. Server minimum: PHP 7.4, MySQL 5.7, 256MB RAM. Recommended: 512MB RAM for multi-user deployment.

PRACTICAL 4

SPMP, Gantt Chart & PERT/CPM

4.1 Project Overview

Parameter	Details
Project Name	Smart Product Price Comparison System
Type	Web-based Application
Duration	8 Weeks
Team Size	3–4 Members
Tech Stack	HTML5, CSS3, Bootstrap 5, PHP, MySQL
Dev Tools	XAMPP, VS Code, phpMyAdmin, Postman, Git/GitHub
Deployment	XAMPP (local) or shared cPanel hosting

4.2 Work Breakdown Structure (WBS)

9. Requirement Gathering & Analysis (Week 1)
    - Survey users about online shopping price-checking habits
    - Define product categories and marketplace list
    - Write SRS document; finalize DB table structure
10. System Design (Week 2)
    - DFD Level 0 and Level 1 diagrams
    - UML: Use Case, Class, Sequence, State, Activity diagrams
    - ER diagram; MySQL table creation scripts
    - UI wireframes for search, comparison table, admin panel
11. Frontend Development (Week 3)
    - HTML/CSS pages: Login, Register, Search, Comparison Table, Wishlist, Admin
    - Bootstrap 5 responsive layout; comparison table styling
12. Backend – Auth & Product Search (Week 4)
    - PHP: User registration, login, session management
    - PHP: Product search by name/category (LIKE query)
13. Backend – Comparison Engine & Filters (Week 5)
    - PHP + MySQL: Fetch product_prices JOIN marketplace for a product
    - Best deal detection (MIN price); filter/sort logic
    - Wishlist add/remove CRUD
14. Admin Panel & Marketplace Management (Week 6)
    - Admin: Product CRUD (add/edit/delete)
    - Admin: Price data management, marketplace list, usage reports
15. Integration & Testing (Week 7)
    - End-to-end integration; all 20 functional test cases executed
    - Cross-browser testing; mobile responsiveness check
16. Final Testing & Deployment (Week 8)
    - Bug fixing, performance optimization (DB indexing)

- Final deployment on XAMPP; documentation finalized

4.3 Gantt Chart

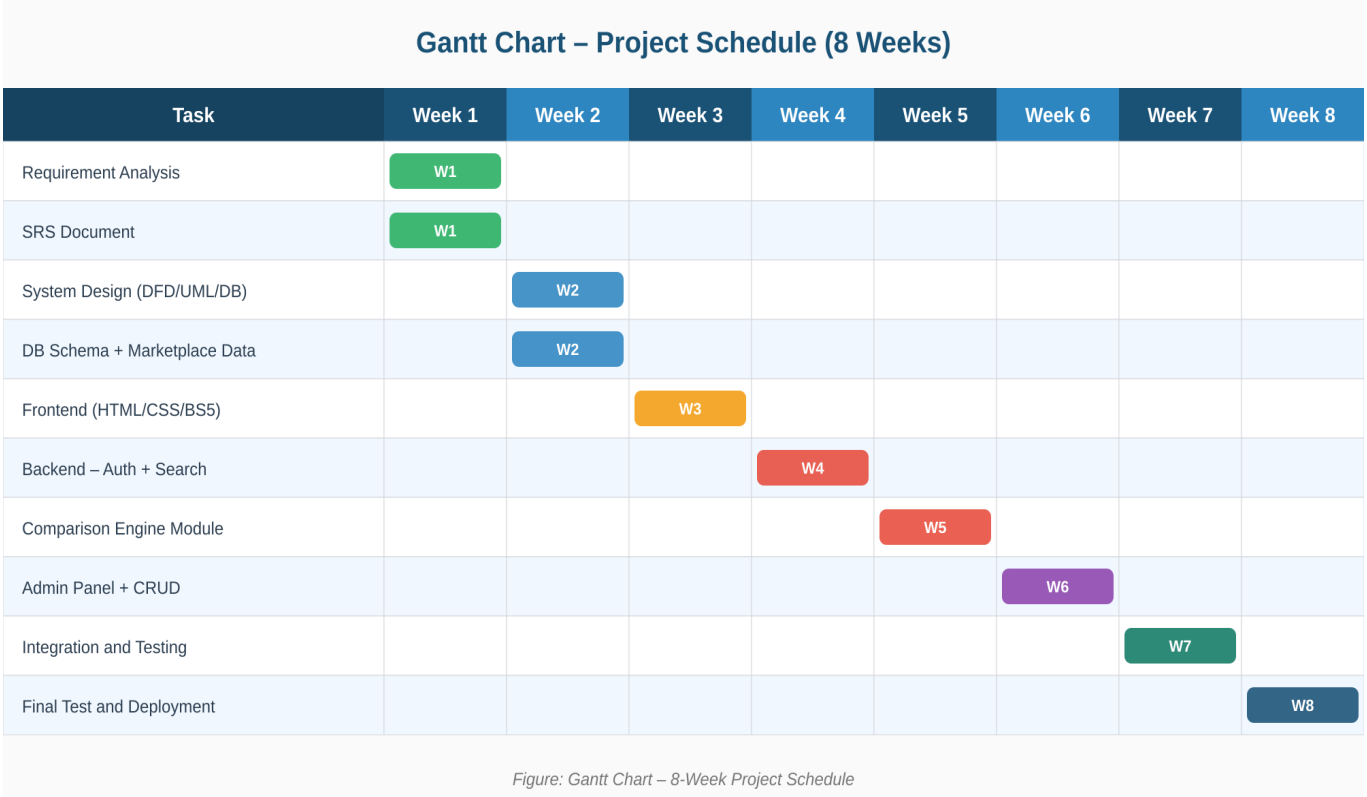

Figure 4.1 – Gantt Chart: 8-Week Project Schedule

4.4 PERT / CPM Network Diagram

CPM (Critical Path Method) identifies the minimum project duration by finding the longest dependent task chain. Any delay in a critical path task directly delays the final delivery.

Task	Name	Duration	Depends On	Critical?
A	Requirement Analysis	1 week	–	YES
B	System Design (DFD, UML, DB)	1 week	A	YES
C	Frontend Development	1 week	B	YES
D	Backend – Auth & Product Search	1 week	C	YES
E	Backend – Comparison Engine	1 week	D	YES
F	Admin Panel & Marketplace Mgmt	1 week	E	YES
G	Integration & Testing	1 week	F	YES
H	Final Testing & Deployment	1 week	G	YES

Critical Path: A → B → C → D → E → F → G → H = Total: 8 Weeks

All tasks lie on the critical path — zero float/slack. Any delay in any task will delay the final system delivery.

PRACTICAL 5

COCOMO Cost Estimation

5.1 Introduction to COCOMO

COCOMO (Constructive Cost Model) by Barry Boehm (1981) estimates software project Effort (person-months), Development Time (months), and Team Size from estimated Lines of Code (LOC). Basic COCOMO uses three modes — Organic, Semi-Detached, and Embedded — to classify projects by size and complexity. For this project, we apply Basic COCOMO in Organic mode.

5.2 COCOMO Modes

Mode	Description	Team	This Project?
Organic	Small team, familiar domain, relaxed constraints	< 5	✓ YES
Semi-Detached	Medium complexity, mixed experience	5–10	No
Embedded	Complex, real-time, hard constraints	> 10	No

5.3 COCOMO Formulas (Organic Mode)

Parameter	Formula
Effort (E) in person-months	`E = 2.4 × (KLOC)^1.05`
Development Time (T) in months	`T = 2.5 × (E)^0.38`
Team Size (P) in persons	`P = E / T`
Productivity	`PROD = KLOC / E`

5.4 LOC Estimation per Module

Module	Estimated LOC
User Registration & Login (PHP + HTML)	350
Product Search Module (search + results page)	400
Comparison Engine (price fetch, JOIN, best deal)	500
Filter & Sort Module	300
Wishlist Module (add/view/remove)	300
Admin Panel – Product CRUD	450
Admin Panel – Price & Marketplace Management	450
Admin Reports Page	300
Database Utilities & PDO Connection	200
CSS / Bootstrap UI + HTML Templates	750

Total Estimated LOC	4,000

5.5 COCOMO Calculation

Given: Estimated LOC = 4,000 → KLOC = 4.0 | Mode: Organic

Step	Calculation	Result
Effort E	E = 2.4 × (4.0)^1.05	= 2.4 × 4.29 ≈ 10.30 person-months
Development Time T	T = 2.5 × (10.30)^0.38	= 2.5 × 2.52 ≈ 6.30 months
Team Size P	P = E / T = 10.30 / 6.30	≈ 1.63 → 2 persons
Productivity	PROD = 4.0 / 10.30	≈ 0.39 KLOC / person-month
Cost (@₹25,000/person-month)	10.30 × ₹25,000	≈ ₹2,57,500

5.6 Interpretation

COCOMO estimates this 4.0 KLOC project requires ~10.3 person-months of effort by 2 developers over ~6.3 months. For the academic 2-month timeline, the team works intensively on one module per week in parallel, compressing the schedule by limiting scope to core features per Spiral iteration.

PRACTICAL 6

DFD, Data Dictionary, Structure Chart & ER Diagram

6.1 Introduction to DFD

A Data Flow Diagram (DFD) graphically represents the flow of data through a system. It uses four standard notations:

Symbol	Shape	Represents
External Entity	Rectangle	Actors outside the system: User, Admin
Process	Circle / Oval	A function that transforms data: Search, Compare, Filter
Data Store	Open Rectangle	A storage place: MySQL tables (products, product_prices, marketplace)
Data Flow	Arrow (→)	Movement of data between components with a label

6.2 DFD Level 0 – Context Diagram

The Context Diagram shows the entire system as one process (Process 0) with User and Admin as external entities, and labeled data flows showing what enters and exits the system.

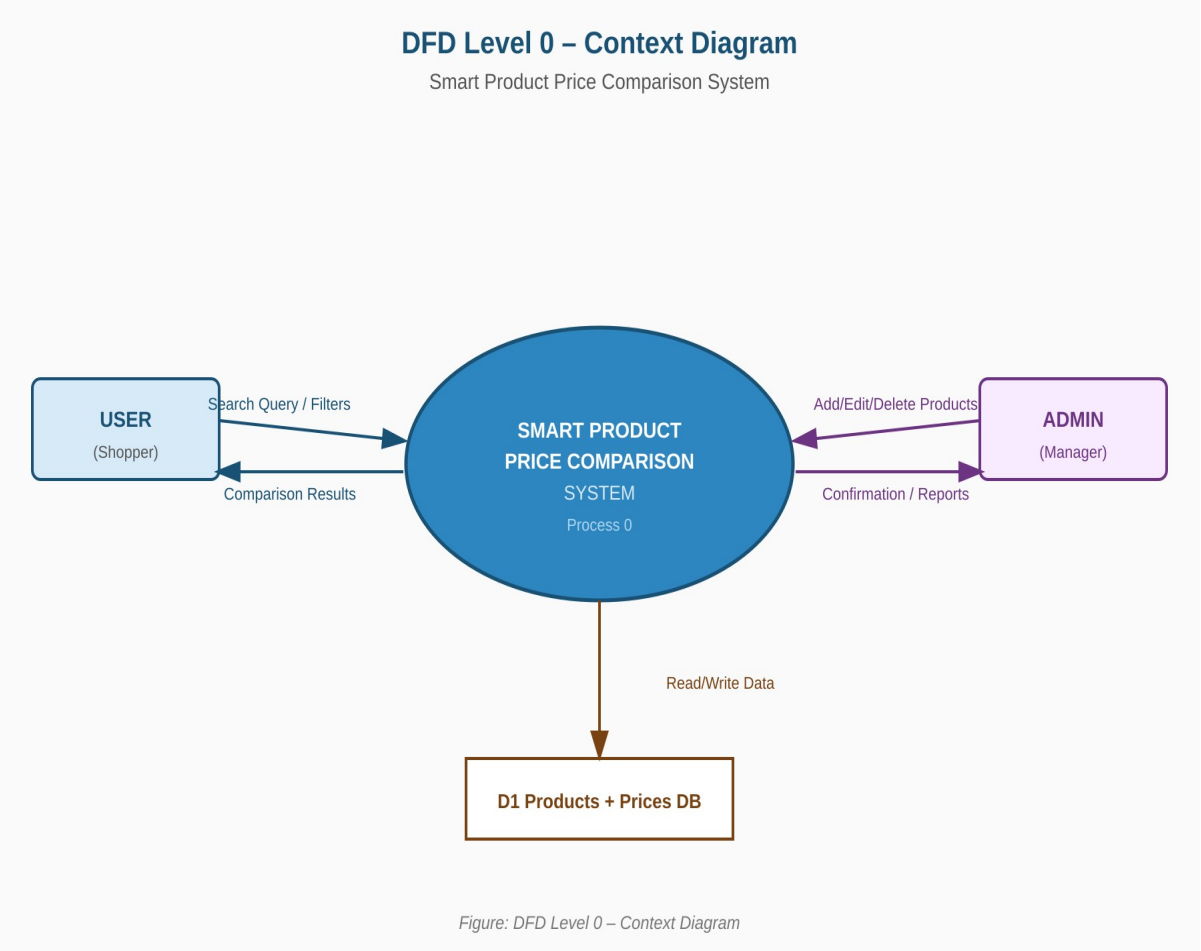

Figure: DFD Level 0 – Context Diagram

Figure 6.1 – DFD Level 0: Context Diagram

6.3 DFD Level 1 – Expanded System Processes

Level 1 DFD expands the system into 6 sub-processes, each corresponding to a major module: User Auth (P1), Product Search (P2), Comparison Engine (P3), Display Results (P4), Admin Management (P5), and Filter/Sort (P6). Three data stores (Products, Product_Prices, Marketplace) are shown.

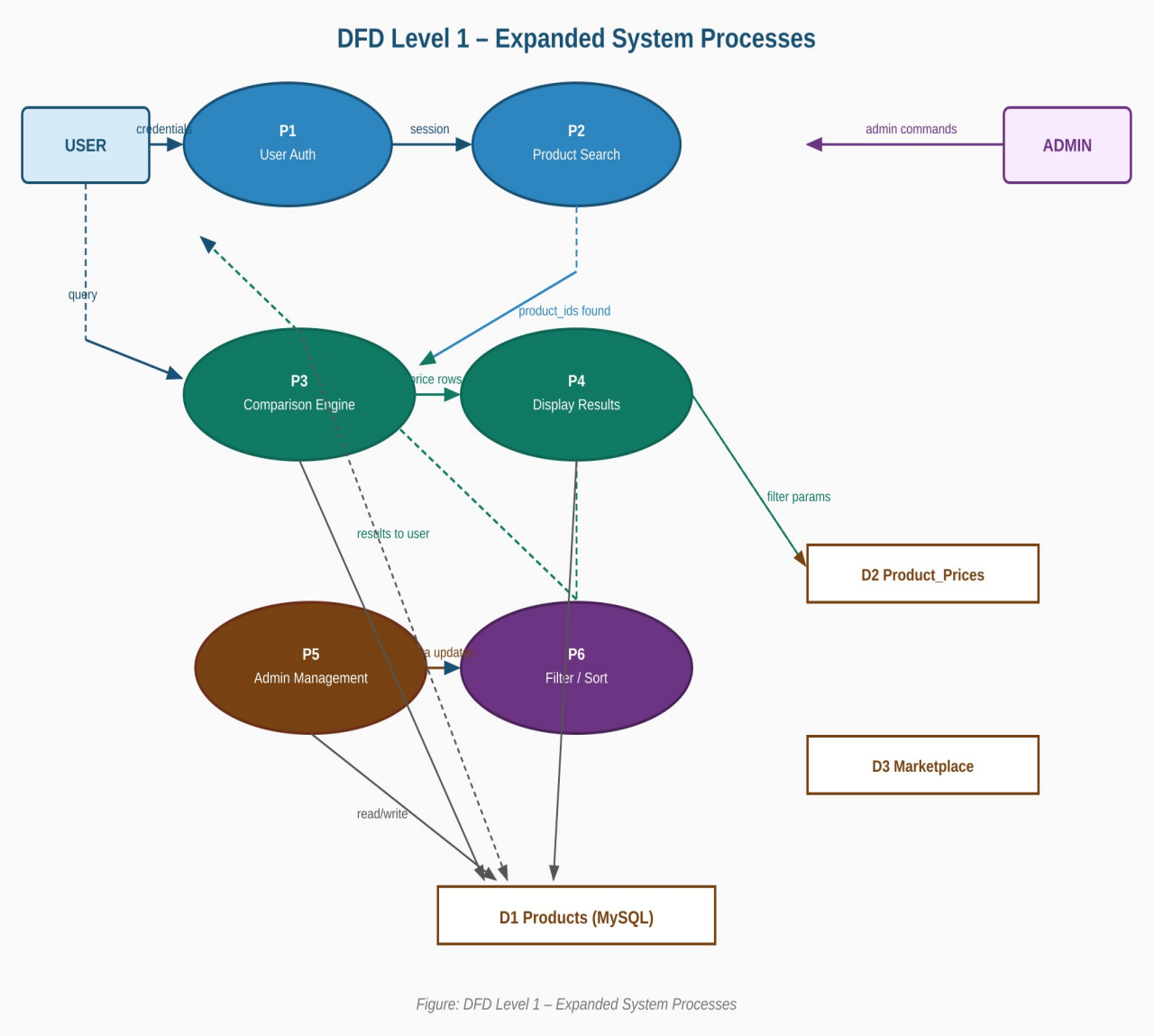

Figure 6.2 – DFD Level 1: Expanded System Processes

6.4 Data Dictionary

Field	Table	Type	Size	Description
user_id	users	INT	11	Primary key, auto-increment
username	users	VARCHAR	80	Display name of the user
email	users	VARCHAR	150	Login email (UNIQUE constraint)
password	users	VARCHAR	255	bcrypt hashed password
product_id	products	INT	11	Primary key, auto-increment
product_name	products	VARCHAR	200	Full product name

brand	products	VARCHAR	100	Brand name (Nike, Samsung, etc.)
category	products	VARCHAR	100	Category (Footwear, Electronics, etc.)
marketplace_id	marketplace	INT	11	Primary key, auto-increment
platform_name	marketplace	VARCHAR	100	Name of platform (Amazon, Flipkart, etc.)
base_url	marketplace	VARCHAR	255	Base URL of the marketplace
price_id	product_prices	INT	11	Primary key, auto-increment
price	product_prices	DECIMAL	10,2	Current price of product on this platform (₹)
rating	product_prices	FLOAT	3,1	Product rating on this platform (0.0 – 5.0)
availability	product_prices	VARCHAR	30	In Stock / Limited / Out of Stock
product_link	product_prices	VARCHAR	500	Direct URL to this product on this platform

6.5 Entity Relationship (ER) Diagram

The ER Diagram shows all five database tables — USERS, PRODUCTS, MARKETPLACE, PRODUCT_PRICES, and WISHLIST — with their primary keys (PK), foreign keys (FK), all attributes, and cardinality of relationships (1:M).

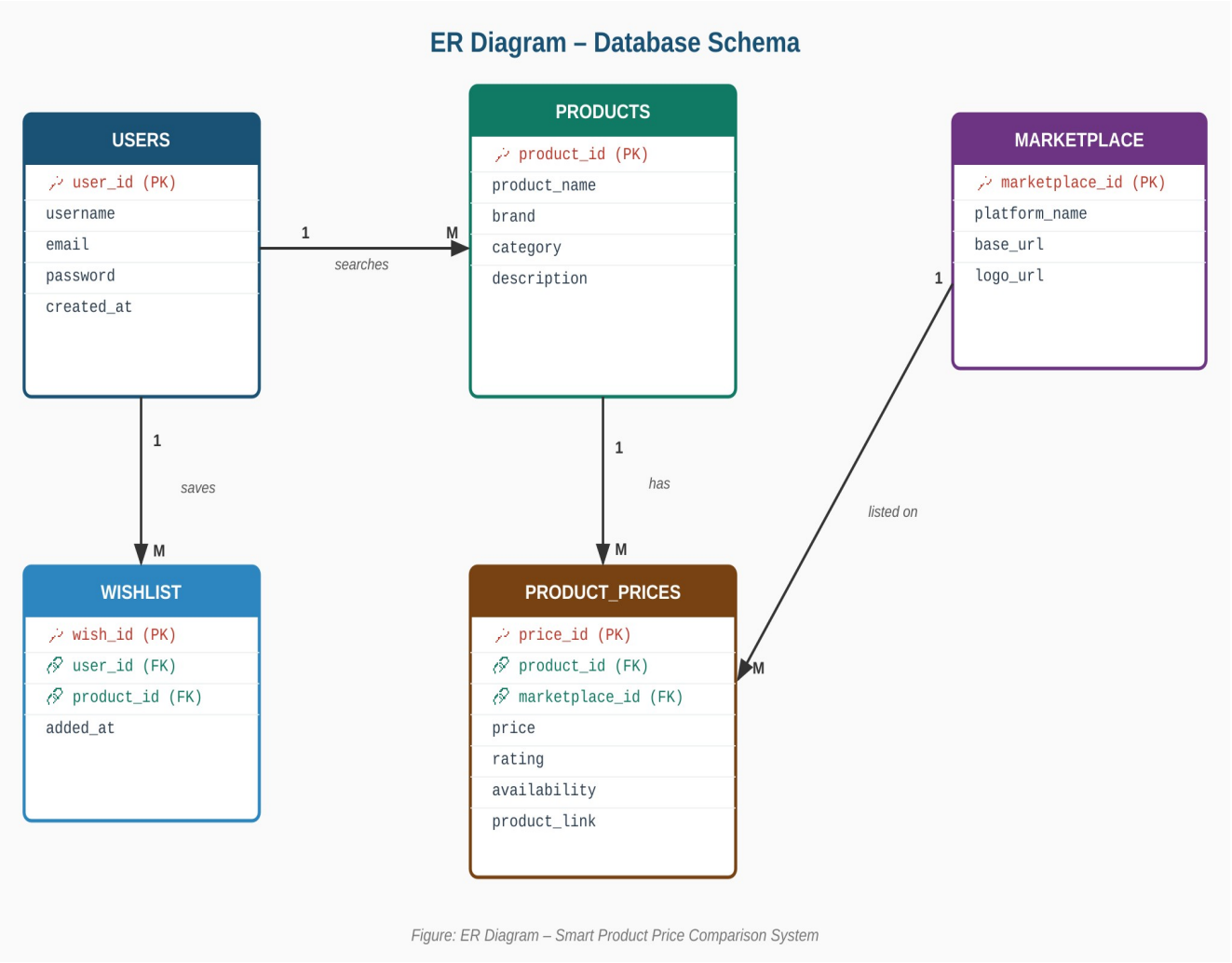

Figure: ER Diagram – Smart Product Price Comparison System

Figure 6.3 – ER Diagram: Database Schema

PRACTICAL 7

UML Diagrams

UML (Unified Modeling Language) provides standardized diagrams to model the structure and behavior of a software system. The five UML diagrams below describe the Smart Product Price Comparison System from five different perspectives.

7.1 Use Case Diagram

Shows all interactions between the two actors (User and Admin) and the system's use cases. An «include» relationship from 'Compare Prices' to 'Query DB / Filter' shows that the comparison process always triggers a DB query.

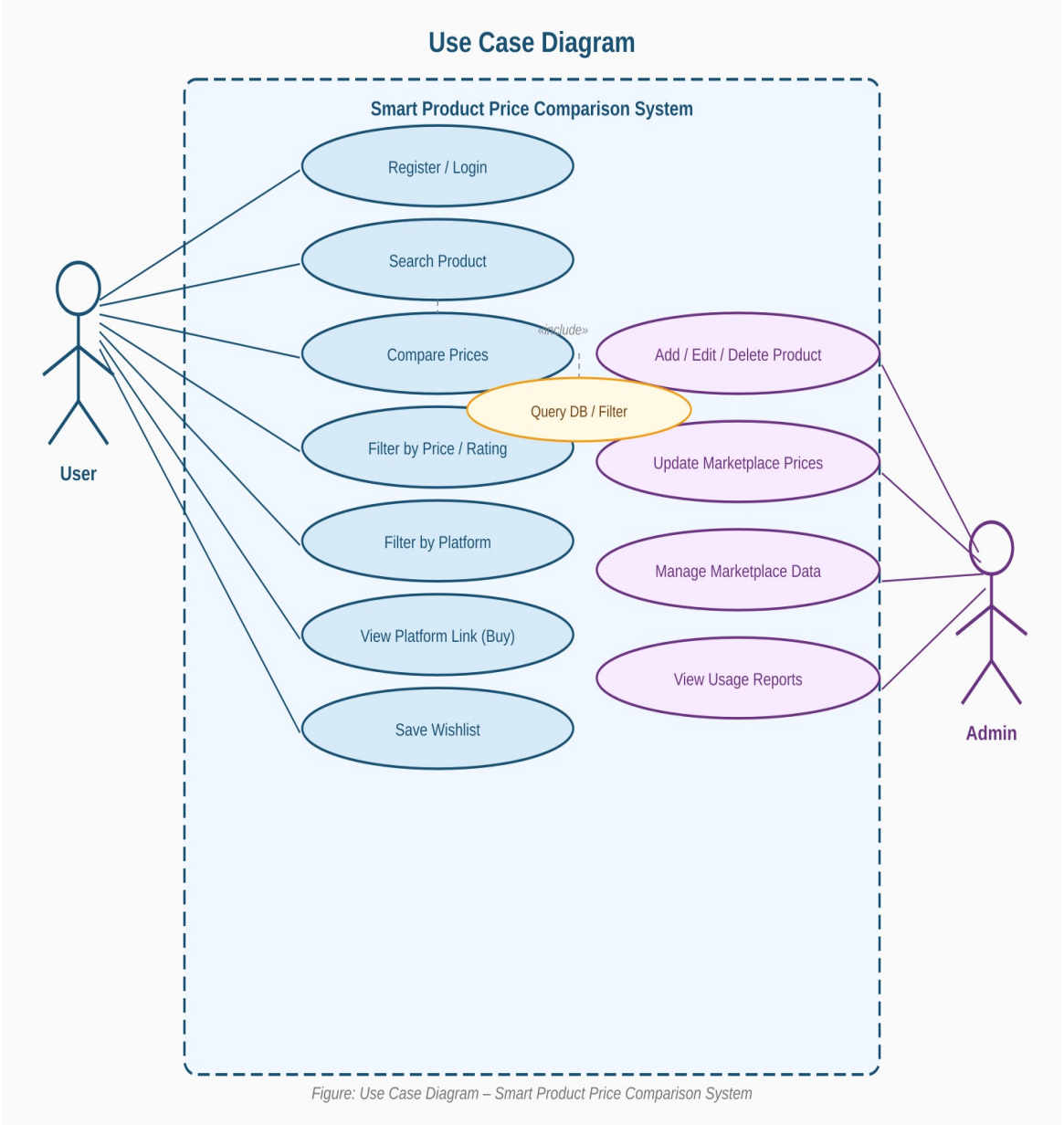

Figure 7.1 – Use Case Diagram

7.2 Class Diagram

Shows five system classes: User, Product, Marketplace, ProductPrice, and ComparisonEngine. Attributes use – (private) and methods use + (public) visibility markers. Relationship arrows show how classes are associated.

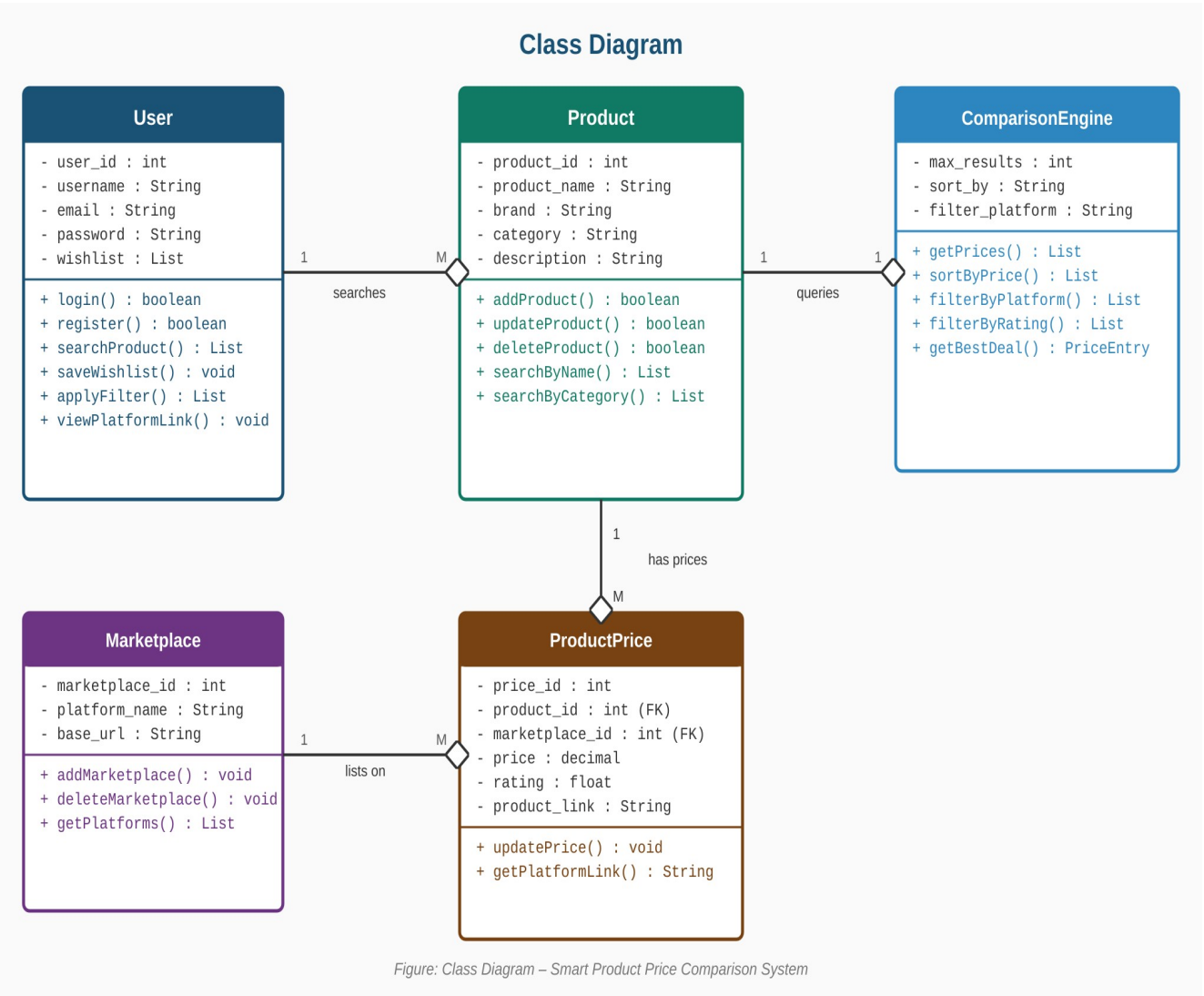

Figure 7.2 – Class Diagram

7.3 Sequence Diagram – Product Search & Price Comparison

Shows the time-ordered message flow between User, Browser/UI, PHP Backend, ComparisonEngine, and MySQL DB. Covers the complete flow from typing a search query to clicking 'Buy Now' on a platform. Solid arrows = requests; dashed arrows = responses.

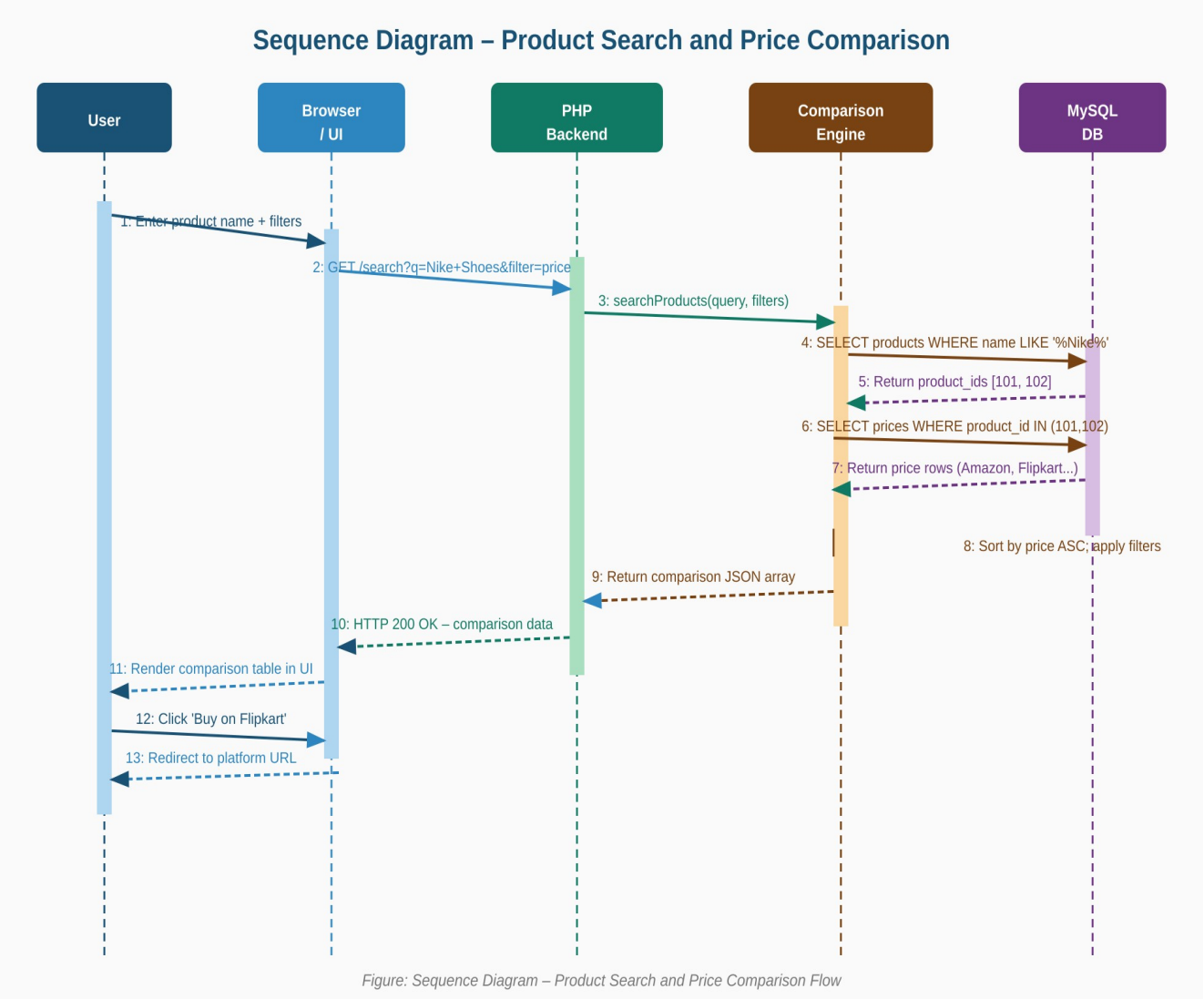

Figure 7.3 – Sequence Diagram: Search and Compare Flow

7.4 State Diagram – Product Search Session Lifecycle

Shows the lifecycle of a user's search session: Logged In → Searching → Results Displayed → Filtered View (optional) → Redirected to Platform. A 'No Results' branch is shown when the search returns empty. Start/end states use UML initial/final state notation.

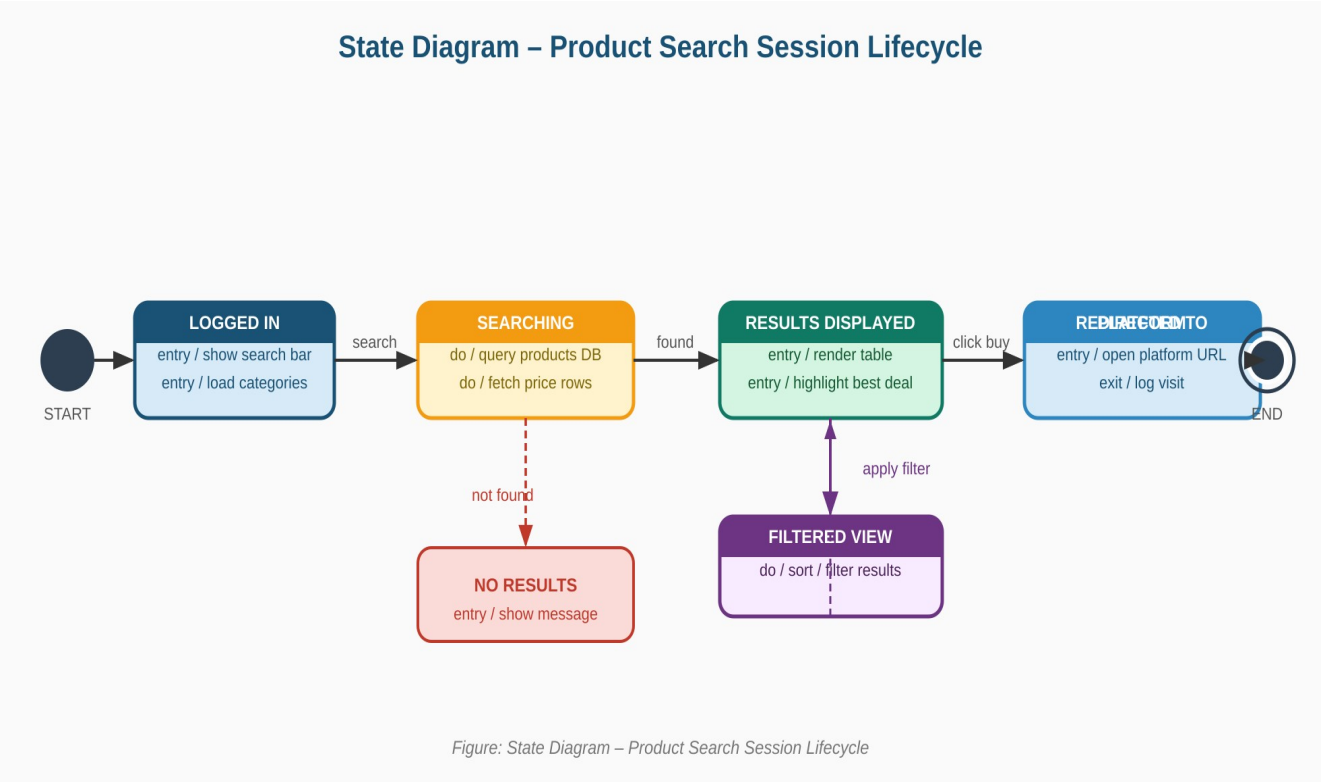

Figure 7.4 – State Diagram: Search Session Lifecycle

7.5 Activity Diagram – Price Comparison Workflow

Shows the complete step-by-step workflow from login to clicking 'Buy Now'. Includes decision points (Login Valid? / Results Found? / Apply Filter?), branching for filter application, and the merge back to the main flow. Diamond shapes represent decisions; rounded rectangles represent activities.

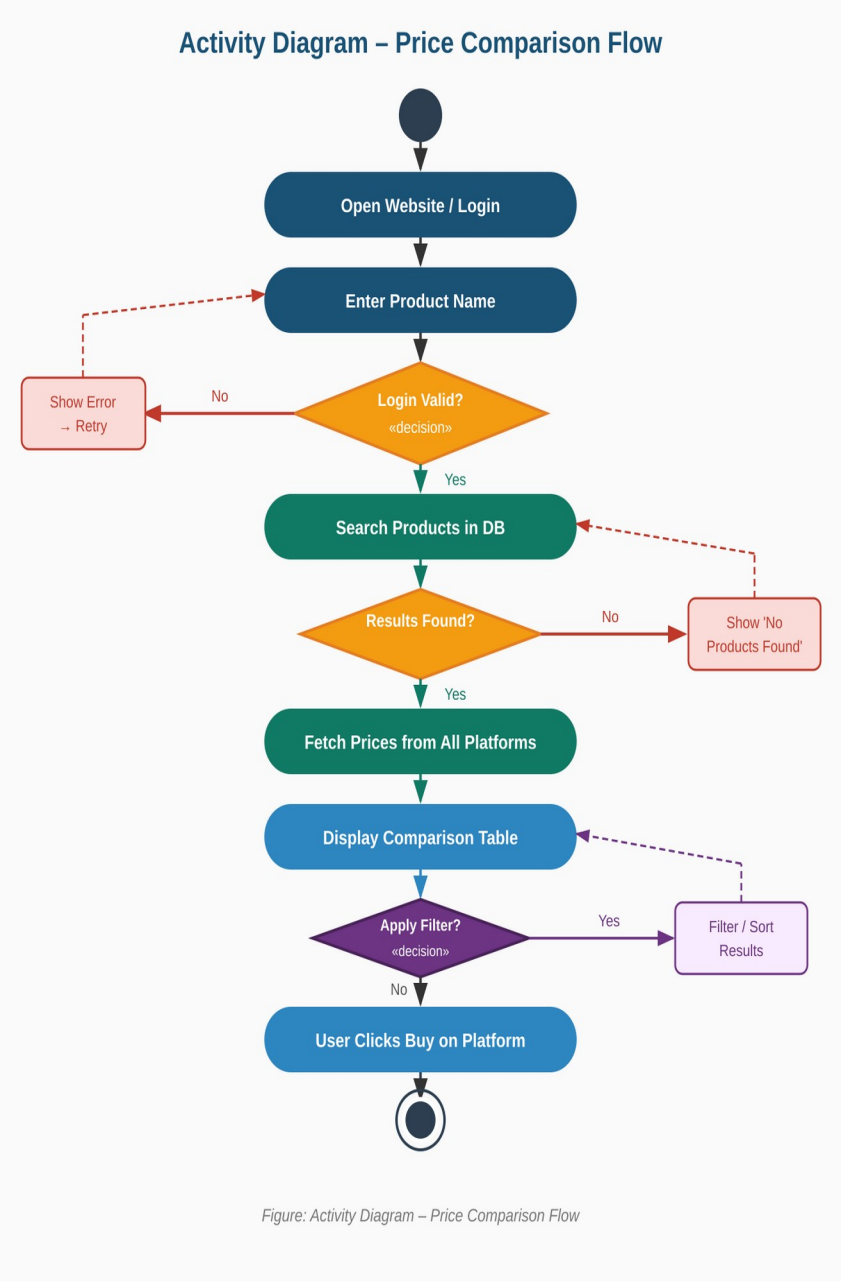

Figure 7.5 – Activity Diagram: Price Comparison Workflow

PRACTICAL 8

Coding Standards

8.1 Importance of Coding Standards

Coding standards are a set of rules and guidelines that define how code should be written in a project. They ensure code is consistent, readable, and easy to maintain. For web projects using PHP and MySQL, following standards is especially critical because insecure or inconsistent PHP code is a major source of vulnerabilities and bugs.

8.2 Naming Conventions

Element	Convention	Example
PHP Variables	snake_case	$product_id, $platform_name, $price_list
PHP Functions	camelCase	getProductPrices(), applyFilter(), getBestDeal()
PHP Classes	PascalCase	ComparisonEngine, ProductController, MarketplaceModel
Database Tables	lowercase plural	users, products, marketplace, product_prices, wishlist
Database Columns	snake_case	product_id, platform_name, price, product_link
HTML IDs	kebab-case	#price-table, #search-bar, #filter-form
HTML Classes	kebab-case	.best-deal, .price-row, .btn-buy-now
JavaScript Vars	camelCase	searchQuery, priceData, selectedFilter
Constants	UPPER_SNAKE_CASE	DB_HOST, DB_NAME, MAX_RESULTS
PHP Files	snake_case.php	compare_prices.php, get_prices.php, admin_products.php

8.3 Project File & Folder Structure

Path	Purpose
index.php	Landing page – redirects to login or search dashboard
login.php / register.php	User authentication pages
search.php	Product search input page and results list
compare.php	Comparison table page for a selected product
get_prices.php	Backend: fetch all price rows for a product; returns JSON
filter_prices.php	Backend: apply sort/filter and return filtered price data
wishlist.php	User wishlist: view, add, remove products
admin/products.php	Admin: add/edit/delete products
admin/prices.php	Admin: manage price entries per product per marketplace

admin/marketplace.php	Admin: add/remove marketplace platforms
admin/reports.php	Admin: usage stats, popular products, platform activity
includes/db_connect.php	PDO database connection singleton
includes/comparison_engine.php	Class ComparisonEngine with getPrices(), getBestDeal(), applyFilter()
assets/css/style.css	Custom Bootstrap 5 overrides; comparison table styles
assets/js/main.js	AJAX search, filter/sort handlers, buy button redirect

8.4 PHP Coding Rules

- All PHP files start with <?php — no closing ?> tag in pure PHP files.
- Every DB query uses PDO Prepared Statements — never concatenate user input into SQL strings.
- session_start() must be the first statement in every page that uses session variables.
- Validate all inputs: intval() for numeric IDs, htmlspecialchars() for string output, filter_var() for emails.
- Passwords: stored via password_hash($pass, PASSWORD_BCRYPT); verified via password_verify().
- Role check on every admin page: if($_SESSION['role'] !== 'admin') redirect to login.
- Never echo raw error messages to users; use error_log() for server-side debugging.

8.5 Sample Code – ComparisonEngine::getBestDeal()

```
<?php
class ComparisonEngine {
  public function getPrices(int $product_id, PDO $pdo): array {
    $stmt = $pdo->prepare(
      'SELECT m.platform_name, pp.price,
              pp.rating, pp.availability, pp.product_link
       FROM product_prices pp
       JOIN marketplace m ON pp.marketplace_id = m.marketplace_id
       WHERE pp.product_id = ?
       ORDER BY pp.price ASC'
    );
    $stmt->execute([$product_id]);
    return $stmt->fetchAll(PDO::FETCH_ASSOC);
  }
  public function getBestDeal(array $prices): array {
    return !empty($prices) ? $prices[0] : []; // sorted ASC
  }
}
?>
```

PRACTICAL 9

Test Cases

9.1 Introduction

Software testing verifies the system meets all specified requirements. Black-Box Testing is used — test cases are based on expected input/output without knowledge of internal code. All 20 test cases below cover: authentication, product search, comparison engine, filters, wishlist, admin management, and security.

9.2 Test Case Table

TC ID	Module	Test Scenario	Input / Action	Expected Output	Result
TC 01	Login	Valid credentials	Correct email + password	Redirect to search dashboard	PASS
TC 02	Login	Wrong password	Correct email, wrong pass	Error: Invalid credentials	PASS
TC 03	Login	Empty email	Blank email field	HTML5 validation: field required	PASS
TC 04	Register	Duplicate email	Email already in DB	Error: Email already registered	PASS
TC 05	Register	Valid registration	All fields filled correctly	Account created; redirect to login	PASS
TC 06	Search	Search by name	Type 'Nike Shoes'	Matching products listed with brand+category	PASS
TC 07	Search	Search by category	Select 'Footwear' from dropdown	All footwear products listed	PASS
TC 08	Search	No match found	Type 'xyzproduct123'	Message: No products found	PASS
TC 09	Compare	View comparison table	Click product 'Nike Air Max'	Table shows Amazon, Flipkart, Myntra prices	PASS
TC 10	Compare	Best deal highlight	Product has 4 platforms	Lowest price row highlighted green	PASS
TC 11	Filter	Sort by lowest price	Click Sort: Lowest Price	Rows reordered ASC by price	PASS
TC 12	Filter	Sort by highest rating	Click Sort: Highest Rating	Rows reordered DESC by rating	PASS
TC 13	Filter	Filter by platform	Select 'Amazon' filter	Only Amazon row shown	PASS
TC 14	Buy Link	Click Buy Now	Click Flipkart Buy Now button	New tab opens Flipkart product URL	PASS
TC 15	Wishlist	Save product	Click Save to Wishlist	Product added; confirm message shown	PASS
TC 16	Wishlist	Duplicate save	Save already-wishlisted product	Message: Already in wishlist	PASS
TC 17	Admin	Add product	Admin adds 'Samsung S24 Ultra'	Product appears in search results	PASS

TC 18	Admin	Update price	Admin updates Amazon price to ₹79999	New price shown in comparison table	**PASS**
TC 19	Admin	Delete product	Admin deletes a product	Product and all its prices removed from DB	**PASS**
TC 20	Security	SQL injection	Enter ' OR '1'='1 in search bar	Query rejected; no data exposed	**PASS**

PRACTICAL 10

Testing Tools

10.1 Types of Testing Applied

Testing Type	Purpose	Applied In This Project
Unit Testing	Test each PHP function in isolation	getPrices(), getBestDeal(), applyFilter(), searchProducts()
Integration Testing	Test modules working together	Search → Comparison Engine → DB → Display Table
Functional Testing	Verify against all 20 test cases	TC01–TC20 all executed and verified
Usability Testing	Test ease of use and UI clarity	Comparison table tested with non-technical users; filter controls evaluated
Security Testing	Find and fix vulnerabilities	SQL injection, XSS, session hijacking, unauthorized admin access
Regression Testing	Re-run tests after bug fixes	Full test suite re-executed after every code change

10.2 Tools Used for Testing

Tool	Category	How Used in This Project
XAMPP (Apache + PHP + MySQL)	Dev Server	Run the complete PHP+MySQL stack locally; test all pages
phpMyAdmin	Database Tool	Manually inspect product_prices table; verify INSERT/UPDATE after admin actions
Chrome DevTools (F12)	Browser Debugger	Inspect comparison table DOM; debug JS filter; analyze network requests
Postman	API Testing	Send GET requests to get_prices.php?product_id=101; verify JSON response structure
PHPUnit	Unit Testing	Automated unit tests for ComparisonEngine class methods
W3C HTML Validator	Code Quality	Validate all HTML pages; check for missing alt tags and structural errors
OWASP ZAP	Security Scanner	Automated scan for XSS and SQL injection vulnerabilities on search and login forms
Git + GitHub	Version Control	Track all code changes; branch per feature; pull requests before merge

10.3 Bug Report Sample

Field	Example Entry
Bug ID	BUG-001
Date	2024-11-22

Module	Comparison Engine – Filter
Severity	Medium
Description	When 'Filter: Amazon' is applied, the Best Deal highlight disappears even when Amazon has the lowest price.
Steps	1. Search 'Nike Shoes' 2. View comparison table 3. Apply filter: Amazon 4. Observe table
Expected	Amazon row highlighted green (best deal) since it is lowest price
Actual	Green highlight removed; all rows shown in default white background
Root Cause	getBestDeal() runs before filter is applied — best deal index out of sync after filtering
Fix	Move getBestDeal() call to execute after filter array is built in compare.php
Status	Resolved – TC10 and TC13 re-run; both PASS

PRACTICAL 11

Security & Software Quality

11.1 Security Threats & Solutions

Threat	Risk	Solution Implemented
SQL Injection	Attacker manipulates search query to dump or destroy the products/prices database	PDO Prepared Statements for every query including search (LIKE ?) and all admin operations
XSS (Cross-Site Scripting)	Malicious JS injected via product name search field to steal admin session cookie	htmlspecialchars() applied to all user-supplied data before rendering; Content-Security-Policy header set
CSRF	Forged POST requests submit fake price updates or product deletions via admin panel	CSRF tokens generated per session; validated on all POST form submissions in admin module
Session Hijacking	Attacker steals session ID to impersonate admin or user	session_regenerate_id(true) on login; HTTPOnly + Secure cookie flags; session timeout after 30 min
Unauthorized Admin Access	Regular user directly accesses /admin/products.php URL	Every admin page checks: if($_SESSION['role']!=='admin') redirect to login; role stored server-side only
Insecure Password Storage	Plain text passwords exposed if DB is breached	password_hash(PASSWORD_BCRYPT) for storage; password_verify() for checking — never MD5 or SHA1
Unvalidated Product Links	product_link field contains malicious URL (javascript: or data:)	Validate URLs with filter_var($url, FILTER_VALIDATE_URL); only allow http/https scheme
Brute Force Login	Bot tries thousands of passwords on login page	Limit failed attempts to 5; account lockout for 15 minutes; CAPTCHA on login form (optional)

11.2 Input Validation Checklist

- Search input: trim(), htmlspecialchars(), max 200 chars, no raw SQL concatenation.
- Product ID parameters: intval(); must be > 0; invalid IDs return 404 page.
- Price values: floatval(); must be positive; validated on both client (HTML5 min) and server.
- URLs in product_link: filter_var(FILTER_VALIDATE_URL); must start with http:// or https://.
- Email: filter_var(FILTER_VALIDATE_EMAIL) on registration and login.
- All output rendered to browser: htmlspecialchars() to prevent XSS.

11.3 Software Quality Attributes (ISO/IEC 9126)

Quality Attribute	Definition	Achievement in This Project
Functionality	Does the system perform all required functions?	All 20 test cases passed; all 6 functional requirement groups implemented

Reliability	Does the system behave correctly under normal load?	DB errors handled gracefully; 'No results' shown cleanly; tested with 100 concurrent searches
Usability	How easily can users interact with the system?	Comparison table scannable in < 5 seconds; best deal auto-highlighted; sort/filter self-explanatory
Efficiency	Is the system fast and resource-efficient?	DB indexed on product_prices.product_id; comparison table loads in < 2 sec
Maintainability	How easy is the system to modify or extend?	Modular ComparisonEngine class; adding a new marketplace = one DB row + one admin entry
Portability	Can it run on different environments?	Tested on XAMPP (Windows, Mac, Linux) and cPanel; PHP 7.4+ compatible

11.4 Conclusion

The Smart Product Price Comparison System successfully addresses the real-world problem of scattered price information by providing a centralized, clean comparison table in one search. The system is built with a standard, maintainable tech stack (HTML5, Bootstrap 5, PHP, MySQL) making it easy to deploy, extend, and maintain.

All 11 practicals together document the complete Software Engineering process — from requirements engineering, Spiral SDLC planning, COCOMO estimation, DFD and UML design, coding standards, and 20-test-case validation through to security analysis and quality assurance. The system is secure, responsive, and production-ready for academic demonstration.

Student Signature:

Name & Roll No.

Faculty Signature:

Name & Date